AI Receptionist Product Plans

Blueprint & ready-to-paste prompts — 833 Tidyups / bookcleaning.app

Generated August 3, 2026

1. Product Blueprint — AI Receptionist for Jobber Cleaning Companies

A sellable, self-serve version of the 833 Tidyups system — installable by any cleaning company from the Jobber App Marketplace.

---

1. The Product in One Sentence

Any cleaning company using Jobber connects their account, picks a phone number, customizes what the AI receptionist asks callers, and gets live-transcribed calls that turn into bookings on their own Jobber calendar — no missed calls, no manual data entry.

2. What We Already Proved (in the 833 Tidyups app)

The single-company version is live at bookcleaning.app and working end-to-end:

- Real phone calls answered via Twilio, forwarded to the business line while an AI listens
- Live transcription streamed to a dispatcher dashboard in real time
- AI extracts caller name, address, service type, and preferred time during the call
- Two-way Jobber sync: bookings created/edited/canceled flow both directions
- Live cleaner GPS map, staff management, dispatcher access control (Clerk auth)
- Battle-tested plumbing: multi-instance-safe call state, single-use stream tokens, webhook bursts handled

This is the prototype. The product wraps it in multi-company packaging.

3. The Setup Wizard (the "expobuilder" experience)

New companies configure everything through a checklist-style wizard — no technical steps:

1. Create account !’ company workspace created (owner login)
2. Connect Jobber !’ one-click OAuth (same flow we built; the Jobber app just gets authorized against their account)
3. Pick a phone number !’ choose a local number in their area code (provisioned automatically via Twilio), or forward their existing number to it
4. Customize the receptionist — the core differentiator:
    - Business name + greeting script ("Thanks for calling Sunshine Cleaners!")
    - Check the questions the AI should collect: name, address, service type, home size, pets, preferred date, budget, how-did-you-hear
    - Add custom questions & answers ("Do you bring supplies?" !’ their answer)
    - Services & price ranges the AI can quote
    - Where to ring through: their business/cell number
5. Invite the team !’ dispatchers and cleaners get their own logins
6. Go live !’ test call button verifies everything before launch

4. What Must Change vs. the Single-Company App

Area — Today (833 Tidyups) — Product version

Companies — One, hardcoded — Many — every table keyed by company; strict data isolation

Jobber — One connected account — Each company OAuths their own; webhooks routed by Jobber account ID

Phone — One Twilio number — One number per company (Twilio subaccounts), calls routed by dialed number

AI script — Fixed prompts — Driven by each company's wizard answers (questions, greeting, services)

Logins — One Clerk app, email allowlist — Clerk Organizations — each company is an org with owner/dispatcher/cleaner roles

Branding — 833 Tidyups — Neutral product brand + each company's name in their own dashboard

Billing — None — Subscription (Stripe) — e.g. free trial !’ monthly per-company plan

5. Jobber Marketplace Requirements (verified from Jobber's docs)

- Submit via Developer Center !’ Manage Apps !’ "Request a review"
- Before submitting: 2FA enabled on the developer account, app logo uploaded, pre-submission checklist completed
- Listing needs: app name, developer name, description, features/benefits, gallery screenshots (Manage App URL optional)
- Jobber tests the app themselves; high quality bar; they email to coordinate release
- No announcements/press until Jobber moves the app to "published"

6. Suggested Build Order

1. Foundation — multi-company data model, Clerk Organizations, company signup
2. Jobber per-company OAuth — reuse existing flow, store tokens per company, route webhooks
3. Phone provisioning — Twilio subaccount + number purchase per company, route inbound calls by dialed number
4. Wizard + configurable AI script — the expobuilder-style setup; AI prompts assembled from company config
5. Dashboard — the existing dispatcher panel, scoped per company
6. Billing — Stripe subscription, trial period
7. Polish + marketplace assets — logo, screenshots, listing copy, demo account for Jobber's reviewers
8. Submit for Jobber review

7. Open Product Decisions (for the owner)

- Product name & brand (can't lean on "833 Tidyups")
- Pricing (monthly flat? per-call? per-seat?) — phone minutes and AI transcription are per-call costs to cover
- Include the cleaner mobile app + GPS map in v1, or dashboard-only first?
- Beta strategy: free for 3–5 friendly cleaning companies before the marketplace listing?

2. Prompt — This Project (833 Tidyups)

Good news first: in THIS project you normally don't need to explain anything. The agent automatically reads replit.md (the project's briefing file — just updated with the full current picture) and its own memory notes before working. Just describe what you want changed in plain words.

If you ever want to be extra sure a new session is fully oriented (for example after a long break), paste this:

You're working on my live business app for 833 Tidyups, a home cleaning company in Edmonton. It's live at https://bookcleaning.app and real customers and staff use it daily, so don't break what's working — test changes before telling me they're done.

What the app does today:

- Dispatcher dashboard with bookings, stats, and a fast new-booking form
- AI live-call panel: calls to (825) 533-4317 are answered by Twilio, forwarded to my business line at (780) 718-5092, and transcribed live on the dashboard while AI pulls out the caller's name, address, and service details. My advertised numbers 833-843-9877 and 587-900-7223 forward into that Twilio number.
- Two-way Jobber sync: bookings flow into Jobber, and changes made in Jobber flow back
- Cleaner mobile app with logins and GPS, feeding a live cleaner map on my dashboard
- Staff management and dispatcher access control
- Public booking website with a contact form

How I like to work:

- I'm not technical — explain things by what they do for my business, not code
- When something's finished, tell me clearly if I need to click Publish to put it on the live site (I know changes don't go live until I publish)
- If you need me to test a phone call, tell me exactly what number to dial and what I should see

Read replit.md and your memory notes before starting. My request: [DESCRIBE WHAT YOU WANT HERE]

---

3. Prompt — New Product App Kickoff

How to use this: create a brand-new Replit App, then paste everything below the line as your very first message. Also attach the file docs/jobber-marketplace-product-blueprint.md from this project (download it and drag it into the chat) — the prompt works alone, but the blueprint adds detail.

I'm building a sellable product: an AI receptionist + dispatch system that any cleaning company using Jobber can install from the Jobber App Marketplace. I already built and battle-tested a single-company version for my own cleaning business (833 Tidyups, live at bookcleaning.app) — this new app is the multi-company product version of it. I'm not technical: explain things by business outcome, not code, and make technical decisions yourself.

The product

A cleaning company signs up, connects their own Jobber account, picks a phone number, and customizes what the AI receptionist says and asks. Then: calls to their number get answered, forwarded to their real phone, transcribed live on their dashboard while AI extracts the caller's name, address, and service details — and bookings sync two-way with their Jobber calendar.

The setup wizard (core experience — think "configurator")

Checklist-style onboarding, no technical steps:

1. Create account !’ company workspace
2. Connect Jobber (one-click OAuth)
3. Pick a local phone number (provisioned automatically via Twilio) or forward their existing number
4. Customize the receptionist: business name + greeting script, checkboxes for what the AI should collect (name, address, service type, home size, pets, preferred date, budget, referral source), custom Q&A pairs ("Do you bring supplies?" !’ their answer), services + price ranges, and the phone number to ring through to
5. Invite dispatchers/cleaners with their own logins
6. Test-call button, then go live

Architecture requirements (multi-company from day one)

- Every company's data fully separated (every table keyed by company)
- Each company OAuths their own Jobber account; Jobber webhooks routed by Jobber account ID
- One Twilio phone number per company (Twilio subaccounts); inbound calls routed by the dialed number
- AI prompts assembled from each company's wizard answers — never hardcoded
- Company logins with owner/dispatcher/cleaner roles (an auth system with organizations)
- Subscription billing (Stripe) with a free trial
- Neutral product branding (not 833 Tidyups); each company sees their own name in their

dashboard

Hard-won technical lessons from the working prototype (don't relearn these the hard way)

- Twilio strips query strings from <Stream> WebSocket URLs. Auth tokens for the audio stream must ride a nested <Parameter name="token"> and be read from the start message's customParameters — a ?token= in the URL never arrives. Synthetic tests pass with query tokens; real calls fail.
- Production runs multi-instance (autoscale): never keep live-call or realtime state in process memory. Use a shared database row + Postgres LISTEN/NOTIFY fan-out for live transcription to dashboards. One-time tokens must be enforced in the database (single-use table with atomic insert), not in memory.
- Jobber's GraphQL API retires pinned versions (old versions start 404ing and break every sync) — pin a current version, note it, and expect to bump it. Creating a booking requires client !’ property !’ request in sequence; "requestEdit" only edits the title.
- Webhook endpoints need their own simple auth (signed query param works — HTTP webhooks keep query strings; only Twilio's WebSocket streams lose them).
- Rate-limit / debounce bursts of Jobber webhooks so you don't hammer their API.

Jobber Marketplace listing (the end goal)

Submitted from my Jobber Developer Center ("Manage Apps" !’ "Request a review"). Requirements: 2FA on my developer account, app logo uploaded, pre-submission checklist, listing copy (name, description, features/benefits) and gallery screenshots. Jobber tests the app themselves and coordinates release timing. Build so a Jobber reviewer can sign up and try it with a demo/test setup.

Build order

1. Multi-company foundation: signup, company workspaces, role-based logins
2. Per-company Jobber OAuth + webhook routing
3. Per-company phone number provisioning + call routing
4. Setup wizard + configurable AI receptionist script
5. Live-call dashboard (transcription + AI extraction), scoped per company
6. Stripe subscription billing with trial
7. Marketplace assets (logo, screenshots, listing copy) + demo account for reviewers

Before you start building

Ask me these one at a time (I haven't decided yet): the product's name/brand, monthly pricing, and whether version 1 includes the cleaner mobile app + GPS map or is dashboard-only. Then start with step 1 of the build order.
